

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 5

Выполнил:

Захматов Юрий

Группа:

K3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

- подключить и настроить rabbitMQ/kafka;
- реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

Ход работы

1. Запуск Kafka

Kafka была запущена через Docker Compose.

2. Добавление зависимости

Во все сервисы, которым необходимо отправлять или получать сообщения (Payment, Booking, Reviews, Property), добавлена зависимость spring-kafka.

3. Настройка сервисов

В application.yml каждого сервиса прописаны адрес Kafka (localhost:9092), настройки сериализации/десериализации JSON и group-id для консьюмеров.

4. Создание событий

Созданы общие DTO для событий:

- PaymentCompletedEvent (оплата завершена)
- PropertyRatingUpdateEvent (обновление рейтинга)

5. Producer - отправка событий

Payment Service после подтверждения оплаты отправляет событие payment.completed в Kafka.

Reviews Service после создания отзыва отправляет событие property.rating.updated в Kafka.

6. Consumer - получение событий

Booking Service слушает топик payment.completed и при получении события обновляет статус бронирования на CONFIRMED, сохраняя ID платежа.

Property Service слушает топик property.rating.updated и при получении события пересчитывает средний рейтинг объявления (обновляет поля avgRating, totalReviews, ratingSum).

7. Создание топиков

Топики создаются автоматически при первом использовании. При необходимости их можно создать явно через NewTopic бин или через консольные команды Kafka.

8. Реализованные сценарии

Сценарий	Producer	Consumer	Топик
Обновление рейтинга	Reviews Service	Property Service	property.rating.updated
Привязка платежа к бронированию	Payment Service	Booking Service	payment.status.updated

9. Преимущества подхода

- **Асинхронность** - сервисы не блокируются в ожидании ответа
- **Устойчивость** - при временной недоступности Consumer'a сообщения сохраняются в Kafka
- **Масштабируемость** - можно добавить несколько Consumer'ов на один топик (например, Notification Service тоже слушает payment.completed для отправки уведомлений)

Вывод

В ходе выполнения работы был подключён и настроен Kafka для асинхронного взаимодействия между микросервисами.